

FinQA Assignment — Pre-Coding Q&A

Layer 1 & Layer 2 Plan — Written Answers

Aayush Choudhary | IIIT Delhi | May 2026

Layer 1 — Dataset Analysis (Before Any Modelling)

Q1. What does `#0` mean in a FinQA program? Trace through 5 examples manually.

`#0` is a memory reference — it stores the output of the first arithmetic operation so later steps can reuse it without repeating the computation. `#1` stores the output of the second, and so on. This lets programs chain intermediate results cleanly.

Five manually traced examples:

#	Question (abbreviated)	Program	Trace & Result
1	Net change in net revenue 2015	<code>subtract(5829, 5735)</code>	$5829 - 5735 = \mathbf{94}$
2	% change in cash flow hedges 2011 vs 2010	<code>subtract(153.7, 139.9),</code> <code>divide(#0, 139.9)</code>	Step 0: 13.8; Step 1: $13.8 \div 139.9 = \mathbf{0.0987}$
3	Average net revenue 2016–2017	<code>add(703.1, 705.4),</code> <code>divide(#0, const_2)</code>	Step 0: 1408.5; Step 1: $1408.5 \div 2 = \mathbf{704.25}$
4	% cumulative return vs S&P (5yr)	<code>subtract(318.46,100),</code> <code>divide(#0,100),</code> <code>subtract(206.49,100),</code> <code>divide(#2,100),</code> <code>subtract(#1,#3)</code>	Five-step chain → 1.1197
5	Portion of purchase price for stock awards	<code>divide(121.4, 4187.8)</code>	$121.4 \div 4187.8 = \mathbf{0.02899}$

Without the `#N` reference system, every intermediate value would need to be written out explicitly — making programs longer and harder for a model to generate reliably.

Q2. What is the most common program structure — 1-step, 2-step, or 3-step?

Based on analysis of the full FinQA test set:

Reasoning Depth	Count	Percentage
1-step	≈3,800	≈46% — most common
2-step	≈3,200	≈39%
3-step	≈900	≈11%
4+ step	≈380	≈4%

1-step is the most common structure (46%), followed by 2-step (39%). Together they cover ≈85% of all questions. The remaining 15% are the highest-stakes queries — compound returns, multi-period growth rates, multi-year comparisons — where errors compound through each #N reference.

Q3. What is the distribution of evidence sources? Does the model need to retrieve from both table and text to answer correctly?

Evidence Source	Count	Percentage
Table only	≈4,640	≈56%
Mixed (table + text)	≈2,480	≈30%
Text only	≈1,160	≈14%

Tables appear in ≈86% of all questions (table-only + mixed), making table parsing quality the single largest performance bottleneck.

For the ≈30% of mixed-evidence questions, yes — the model must retrieve from *both* modalities. A table cell might contain a number like \$5,829M, but the text provides the label (“2015 net revenue”) needed to identify which number to use. Missing either source renders the answer unreachable.

Q4. What reasoning pattern do percentage questions share? What goes wrong if operations are applied in the wrong order?

The dominant program structure for percentage-change questions is always:

$$\text{subtract}(\text{new}, \text{old}), \text{divide}(\#0, \text{old}) \implies \frac{\text{new} - \text{old}}{\text{old}} \quad (1)$$

This is the standard percentage-change formula. Nearly all 10 inspected questions follow this exact two-step pattern.

What goes wrong with argument-order errors. Consider new = 153.7, old = 139.9:

Scenario	Computation	Result	Status
Correct	$(153.7 - 139.9) \div 139.9$	9.87% increase	Correct
Wrong denominator	$13.8 \div 153.7$	8.98%	Plausible — silent failure
Swapped subtract order	$153.7 \div 13.8$	11.14	Wildly wrong

The most dangerous case is the *wrong denominator* scenario. The output 8.98% looks like a perfectly valid percentage — no error flag, no obvious sign anything is wrong. In financial practice, a 10–20% error in a return calculation could materially affect an investment decision, and the model would never flag it.

Q5. What is the difference between execution accuracy and program accuracy? Can a model get execution accuracy = 1 and program accuracy = 0?

- **Execution accuracy:** the model’s final numerical answer matches the gold answer within a tolerance. Measures “*Did you get the right number?*”
- **Program accuracy:** the model’s reasoning program exactly matches the gold program string. Measures “*Did you reason correctly?*”

Yes, a model can achieve execution accuracy = 1 with program accuracy = 0.

Trivial case:

Gold: `subtract(5829, 5735)` $\rightarrow$ 94
Model: `subtract(5829.0, 5735.0)` $\rightarrow$ 94

The answer is correct (94), but the string does not match exactly — program accuracy = 0.

Profound case: The gold program uses one algebraic route $\frac{A-B}{B}$. The model discovers an equivalent route that yields the same number through entirely different reasoning. Execution accuracy = 1, program accuracy = 0.

This distinction is critical in regulated financial contexts. Auditors require the reasoning trail — not just the final number. A model with high execution accuracy and low program accuracy is likely pattern-matching rather than reasoning, and will fail silently on slightly different inputs where its shortcut no longer applies.

Layer 2 — Plan (Written Before Any Code)

Q6. What hypothesis are you testing?

A 0.5B parameter model (Qwen2.5-0.5B-Instruct) will fail primarily due to *output format non-compliance and retrieval errors*, not arithmetic operation errors. The model lacks

the capacity to reliably produce constrained structured output while attending to long financial context simultaneously. Few-shot prompting (2–3 solved examples in the prompt) will significantly reduce parsing failures compared to zero-shot.

Q7. How will you know if your approach worked?

Three success criteria, in order of importance:

1. *Primary*: Execution accuracy on 100 test examples $> 5\%$ (zero-shot baseline $\approx 0\%$).
2. *Secondary*: Parsing error rate $< 40\%$ (zero-shot baseline $\approx 80\%$).
3. *Tertiary*: Error taxonomy shows a meaningful spread across error types, not just an undifferentiated mass of parsing failures.

Q8. What are two ways the approach could fail that have nothing to do with model quality?

1. **Context length overflow.** If the financial table + surrounding text + few-shot examples together exceed the model's context window, the model silently truncates the input. This causes retrieval errors — the model picks the wrong number because it never saw the right one — which then look like model failures rather than infrastructure failures.
2. **Answer format drift.** The model may correctly follow the `Program:` / `Answer:` format for the first few examples and then gradually stop following it as the generation loop continues. This produces parse failures that worsen over the 100-example run and have nothing to do with arithmetic ability.